FREE DEBUGGING KIT FOR SDETs

Playwright Flaky Test Debugging Kit

For SDETs Who Ship Confident Tests

Stop Guessing. Start Fixing.
Ship Confident Tests.

5-Point Diagnosis Checklist • 3 Custom Wait Functions • Locator Cheat Sheet • API Mock Snippets

@shamshutalks

WHAT'S INSIDE

This debugging kit gives you **4 battle-tested resources** to eliminate flaky Playwright tests permanently. Every code snippet uses **Playwright v1.x verified APIs** — copy-paste ready.

[TIP] **Start Here:** If your tests fail intermittently on CI but pass locally, begin with the **5-Point Diagnosis** (page 3). Most flaky tests are caused by hardcoded waits (Point 1) or missing web-first assertions (Point 3). Fix those two first, then tackle the rest.

5-POINT FLAKY TEST DIAGNOSIS

Run these 5 checks in priority order. **Points 1–3 cause 80% of flaky tests.** Each check maps to official Playwright documentation.

01

Hardcoded Waits vs Auto-Waiting

Like a smart traffic light vs a dumb timer — auto-wait waits until the road is clear; hardcoded waits guess a fixed time.

(X) page.waitForTimeout(5000) is a guess — too short on slow CI, too long on fast machines. Textbook flakiness: passes locally, fails on CI.

(OK) Remove ALL waitForTimeout calls. Use locator-based auto-waiting — locator.click() already waits for actionability (visible, stable, receives events, enabled, attached).

02

Locator Specificity and Stability

A locator is like a postal address — '123 Main St' survives renovation; 'the blue building next to the tree' breaks when someone cuts the tree.

(X) CSS classes (.btn-primary), auto-generated IDs (#tsf), and XPath chains break when the UI is redesigned — even without test changes.

(OK) Use getByRole() or getByTestId() — these match user-facing semantics, not DOM structure. Resilience: HIGH vs CSS/XPath LOW.

03

Missing Web-First Assertions

Like a security guard who checks your ID repeatedly until verified — not just glancing once and waving you through.

(X) expect(await locator.isVisible()).toBe(true) is a one-shot check — the element might appear 200ms later but you already checked and got false.

(OK) Use await expect(locator).toBeVisible() — auto-retries until condition met. Never put await BEFORE expect for visibility checks.

5-POINT FLAKY TEST DIAGNOSIS (CONT.)

04

Network Dependency and API Instability

Testing a restaurant's food delivery — if the delivery driver (API) is late, the test fails even though the restaurant (UI) is working perfectly.

(X) Third-party APIs have varying response times (100ms–5000ms+), rate limiting, and different data in staging vs prod. Tests inherit all external unreliability.

(OK) Use page.route() + route.fulfill() to mock external APIs. Control response timing, status codes, and data. Test the UI in isolation from the API.

05

Test Isolation and Shared State

Like each student getting their own exam paper — if students share answers (shared state), one student's mistake affects everyone.

(X) Shared cookies/localStorage, database mutations in one test affecting the next, and tests that depend on execution order create cascading failures.

(OK) Use Playwright's built-in { page } fixture — each test gets its own BrowserContext with isolated cookies, storage, and cache. Never share page instances between tests.

Test Isolation: Shared vs Isolated

```
// [X] BAD — shared state
let sharedPage: Page;
test.beforeAll(async ({ browser }) => {
  sharedPage = await browser.newPage(); // Shared!
});

// [OK] GOOD — isolated per test
test('first', async ({ page }) => {
  await page.goto('/dashboard'); // Own context
});
test('second', async ({ page }) => {
  await page.goto('/dashboard'); // Independent
});
```

CUSTOM WAIT FUNCTION 1/3

waitForNetworkIdle

Wait until all network activity stops for a specified duration. Replaces `page.waitForTimeout()` with a network-aware alternative.

Parameter	Type	Default	Description
page	Page	—	Playwright Page object
options.idleTime	number	500	ms of quiet before resolving
options.timeout	number	30000	Max wait time in ms

waitForNetworkIdle — Full Implementation

```
async function waitForNetworkIdle(
  page: Page,
  options: { idleTime?: number; timeout?: number } = {}
): Promise<void> {
  const { idleTime = 500, timeout = 30_000 } = options;
  await page.waitForFunction(
    (idleMs: number) => {
      return new Promise<boolean>((resolve) => {
        let timer: ReturnType<typeof setTimeout>;
        const check = () => {
          clearTimeout(timer);
          timer = setTimeout(() => resolve(true), idleMs);
        };
        const observer = new PerformanceObserver((list) => {
          for (const entry of list.getEntries()) {
            if (entry.entryType === 'resource') check();
          }
        });
        observer.observe({ entryTypes: ['resource'] });
        check();
      });
    },
    idleTime,
    { timeout }
  );
}
```

Usage Example

```
// Usage example
test('dashboard loads with all widgets', async ({ page }) => {
  await page.goto('/dashboard');
  await waitForNetworkIdle(page, { idleTime: 1000 });
  // All API calls and widget renders have settled
  await expect(page.getByTestId('widget-count')).toHaveText('6');
});
```

CUSTOM WAIT FUNCTION 2/3

waitForSelectorStable

Wait until an element's position and size stop changing. Handles animations, transitions, and re-renders. Useful when you need explicit stability before custom logic.

Parameter	Type	Default	Description
locator	Locator	—	Playwright Locator object
options.stableFrames	number	3	Consecutive frames with same bbox
options.pollInterval	number	50	ms between bounding box checks
options.timeout	number	10000	Max wait time in ms

waitForSelectorStable — Full Implementation

```
async function waitForSelectorStable(
  locator: Locator,
  options: {
    stableFrames?: number;
    pollInterval?: number;
    timeout?: number;
  } = {}
): Promise<void> {
  const { stableFrames = 3, pollInterval = 50, timeout = 10_000 } = options;
  const startTime = Date.now();
  let lastBox: { x: number; y: number; width: number; height: number } | null = null;
  let stableCount = 0;

  while (Date.now() - startTime < timeout) {
    const box = await locator.boundingBox();
    if (box) {
      if (
        lastBox &&
        box.x === lastBox.x &&
        box.y === lastBox.y &&
        box.width === lastBox.width &&
        box.height === lastBox.height
      ) {
        stableCount++;
        if (stableCount >= stableFrames) return;
      } else {
        stableCount = 0;
      }
      lastBox = box;
    }
    await new Promise((r) => setTimeout(r, pollInterval));
  }
  throw new Error(
    `Element did not stabilize within ${timeout}ms`
  );
}
```

Usage Example

```
// Usage — wait for animation to settle before clicking
test('click after animation', async ({ page }) => {
  await page.goto('/animated-menu');
  const item = page.getByRole('menuitem', { name: 'Settings' });
  await item.hover(); // Triggers slide-in
  await waitForSelectorStable(item, { stableFrames: 3 });
  await item.click(); // Safe — animation settled
});
```

CUSTOM WAIT FUNCTION 3/3

waitForApiResponse

Wait for a specific API response and extract the parsed JSON body. Combines page.waitForResponse() with automatic JSON parsing and error handling.

Parameter	Type	Default	Description
page	Page	—	Playwright Page object
urlPattern	string|RegExp	—	Glob or regex to match API endpoint
options.predicate	Function	—	Optional filter on Response object
options.timeout	number	30000	Max wait time in ms

waitForApiResponse — Full Implementation

```
async function waitForApiResponse<T = unknown>(
  page: Page,
  urlPattern: string | RegExp,
  options: {
    predicate?: (response: Response) => boolean;
    timeout?: number;
  } = {}
): Promise<T> {
  const { predicate, timeout = 30_000 } = options;
  const response = await page.waitForResponse(
    (resp) => {
      const urlMatch =
        typeof urlPattern === 'string'
          ? resp.url().includes(urlPattern.replace(/\*/g, ''))
          : urlPattern.test(resp.url());
      return urlMatch && (predicate ? predicate(resp) : true);
    },
    { timeout }
  );
  if (!response.ok()) {
    throw new Error(
      `API ${response.status()} for ${response.url()}`
    );
  }
  return (await response.json()) as T;
}
```

Usage Example

```
// Usage — wait for search API, then assert on results
test('search returns results', async ({ page }) => {
  await page.goto('/search');
  const apiPromise = waitForApiResponse<{ results: Item[] }>(
    page, '**/api/search',
    { predicate: (r) => r.request().method() === 'GET' }
  );
  await page.getByPlaceholder('Search...').fill('playwright');
  await page.keyboard.press('Enter');
  const data = await apiPromise;
  expect(data.results.length).toBeGreaterThan(0);
});
```

LOCATOR STRATEGY CHEAT SHEET

8 official locator strategies ranked by resilience. **Always prefer user-facing attributes** (role, label, text) over CSS/XPath. Source: playwright.dev/docs/locators

Strategy	Best For	Example	Resilience
getByRole()	Buttons, links, headings, inputs	`page.getByRole('button', { name: 'Submit' })`	HIGH
getByTestId()	Complex components, no user-facing attr	`page.getByTestId('login-btn')`	HIGH
getByText()	Non-interactive content (div, span, p)	`page.getByText('Welcome')`	MED
getByLabel()	Form controls with <label>	`page.getByLabel('Email').fill('x@y.com')`	HIGH
getByPlaceholder()	Inputs without visible labels	`page.getByPlaceholder('Search...')`	MED
getByAltText()	<img> and <area> elements	`page.getByAltText('Logo')`	MED
getByTitle()	Elements with title attribute	`page.getByTitle('Issues count')`	MED

Anti-Patterns — What NOT to Do

[X] Anti-Pattern	[OK] Better Alternative	Why
`.btn-primary.episode-actions`	`getByRole('button', { name: '...' })`	CSS classes change with redesigns
`#tsf > div:nth-child(2) > div`	`getByRole('searchbox')`	XPath breaks on DOM restructure
`page.locator('button') [multiple]`	`getByRole('button', { name: 'X' })`	Strict mode throws on multiple
`expect(await loc.isVisible())`	`await expect(loc).toBeVisible()`	Non-retrying = flaky
`page.waitForTimeout(500)`	`Use locator directly — auto-wait`	Fixed delays = slow + flaky

LOCATOR COMPOSITION OPERATORS

Combine locators with **filter()**, **nth()**, **or()**, and **and()** to handle dynamic lists, unpredictable elements, and complex page structures.

Operator	Use Case	Example
filter()	Narrow by text or child element	`page.getByRole('listitem') .filter({ hasText: 'Product 2' }) .getByRole('button', { name: 'Add' })`
nth()	Select by index (use with caution)	`page.getByRole('listitem').nth(1) Caution: Use filter() when possible`
or()	Wait for either element to appear	`const a = page.getByRole('button', { name: 'New' }); const b = page.getByText('Security'); await expect(a.or(b).first()).toBeVisible();`
and()	Combine two criteria	`page.getByRole('button') .and(page.getByTitle('Subscribe'))`
First/last	Match first or last element	`page.getByRole('listitem').first() page.getByRole('listitem').last()`

filter() — Narrow Down in Dynamic Lists

```
// Filter — find specific item in a dynamic list
await page
  .getByRole('listitem')
  .filter({ hasText: 'Product 2' })
  .getByRole('button', { name: 'Add to cart' })
  .click();

// Filter by child element
await page
  .getByRole('listitem')
  .filter({ has: page.getByRole('heading', { name: 'Product 2' }) })
  .getByRole('button', { name: 'Add to cart' })
  .click();
```

or() — Handle Either/Or Scenarios

```
// or() — Handle unpredictable UI outcomes
const newEmail = page.getByRole('button', { name: 'New' });
const securityDialog = page.getByText('Confirm security settings');

// Wait for either one to appear
await expect(newEmail.or(securityDialog).first()).toBeVisible();
if (await securityDialog.isVisible()) {
  await page.getByRole('button', { name: 'Dismiss' }).click();
}
await newEmail.click();
```

JSON MOCK SNIPPETS — SCENARIOS 1–2

6 ready-to-use mock scenarios using `page.route()` + `route.fulfill()`. Source: playwright.dev/docs/mock

Scenario 1: Successful Response (200 + JSON)

Mock a working API that returns user data. Use when the UI depends on API data and you want deterministic test assertions.

Scenario 1 — Success: 200 with JSON

```
await page.route('**/api/v1/users', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({
      users: [
        { id: 1, name: 'Alice', email: 'alice@example.com', role: 'admin' },
        { id: 2, name: 'Bob', email: 'bob@example.com', role: 'viewer' },
      ],
      total: 2,
      page: 1,
    }),
  });
});
```

> Expected: User list renders correctly with 2 entries. Page title shows '2 users'.

Scenario 2: Slow Response (Delayed 2000ms)

Simulate a slow API to test loading states, spinners, and skeleton screens. Verifies UI gracefully handles latency.

Scenario 2 — Slow: Delayed Response

```
await page.route('**/api/v1/dashboard', async (route) => {
  await new Promise((resolve) => setTimeout(resolve, 3000));
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ widgets: [], stats: {} }),
  });
});

// Test loading spinner appears and then disappears
await page.goto('/dashboard');
await expect(page.getByTestId('loading-spinner')).toBeVisible();
await expect(page.getByTestId('loading-spinner')).toBeHidden();
await expect(page.getByTestId('dashboard-content')).toBeVisible();
```

> Expected: Spinner appears during 3s delay, then hides. Dashboard content becomes visible.

JSON MOCK SNIPPETS — SCENARIOS 3–4

Scenario 3: Server Error (500)

Force a 500 error to test error handling UI — error messages, retry buttons, fallback states.

Scenario 3 — Server Error: 500

```
    await page.route('**/api/v1/orders', async (route) => {
      await route.fulfill({
        status: 500,
        contentType: 'application/json',
        body: JSON.stringify({
          error: 'Internal Server Error',
          message: 'Database connection failed',
        }),
      });
    });

    await page.goto('/orders');
    await expect(page.getByText('Something went wrong')).toBeVisible();
    await expect(page.getByRole('button', { name: 'Retry' })).toBeVisible();
```

> Expected: Error message 'Something went wrong' and Retry button visible.

Scenario 4: Empty Response (200 + Empty Array)

Test empty state UI — 'No results found', empty cart, no notifications. Ensures the app handles zero-data gracefully.

Scenario 4 — Empty: 200 with Empty Array

```
    await page.route('**/api/v1/search*', async (route) => {
      await route.fulfill({
        status: 200,
        contentType: 'application/json',
        body: JSON.stringify({ results: [], total: 0 }),
      });
    });

    await page.goto('/search?q=test');
    await expect(page.getByText('No results found')).toBeVisible();
    await expect(page.getByText('Try a different search term')).toBeVisible();
```

> Expected: 'No results found' message and suggestion text visible.

JSON MOCK SNIPPETS — SCENARIOS 5–6

Scenario 5: Conditional Mock (Pass-Through + Fallback)

Mock only the unreliable endpoints while letting stable APIs pass through. Best for partial mocking when most APIs work fine.

Scenario 5 — Conditional: Mock + Pass-Through

```
await page.route('**/api/**', async (route) => {
  const url = route.request().url();
  if (url.includes('/api/v1/payment')) {
    // Payment API is unreliable — mock it
    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify({ success: true, transactionId: 'mock-txn-123' }),
    });
  } else {
    // All other APIs — let them through
    await route.continue();
  }
});
```

> Expected: Payment succeeds with mock data. All other APIs use real responses.

Scenario 6: Modify Real Response

Fetch the real API response, then patch specific fields. Use when you need real data structure but want to control edge cases (e.g., out-of-stock items).

Scenario 6 — Modify: Patch Real Response

```
await page.route('**/api/v1/products', async (route) => {
  const response = await route.fetch(); // Actually call the API
  const json = await response.json();

  // Patch: mark first product as out of stock
  json.products[0].inStock = false;
  json.products[0].stockCount = 0;

  await route.fulfill({
    response, // Preserve original headers/status
    body: JSON.stringify(json),
  });
});
```

> Expected: Product list renders with real data. First product shows 'Out of Stock'.

MOCK VS FIXTURES — DECISION GUIDE

Use this decision table to choose between `page.route()` mocking and Playwright `test fixtures` for each scenario.

Scenario	Approach	Why
Third-party API (payment, auth)	**page.route() mock**	You don't control it, can't guarantee response
Flaky/unstable internal API	**page.route() mock**	Remove external dependency from test
Testing specific error codes (4xx/5xx)	**page.route() mock**	Force error responses on demand
Testing loading states	**page.route() mock + delay**	Control response timing precisely
Stable internal API (you control it)	**Test fixtures + real API**	Test the full stack, not just UI
Database state	**Test fixtures (setup project)**	Seed data before tests, clean up after
Authentication state	**storageState fixture**	Login once, reuse state across tests

Custom Fixture with Mocked API

Custom Fixture with Built-In Mocking

```
// Define a custom fixture for mocked API context
const test = base.extend<{ mockedPage: Page }>({
  mockedPage: async ({ page }, use) => {
    // Set up mocks before every test
    await page.route('**/api/v1/users', route => route.fulfill({
      status: 200,
      json: [{ id: 1, name: 'Test User' }],
    }));
    await use(page);
    // Cleanup happens automatically (route removed with page)
  },
});

test('shows mocked user', async ({ mockedPage }) => {
  await mockedPage.goto('/profile');
  await expect(mockedPage.getByText('Test User')).toBeVisible();
});
```

QUICK REFERENCE CARD

Everything on one page. Tear this out (or bookmark it) for your next debugging session.

01. 5-Point Diagnosis

- ① Remove ALL waitForTimeout calls
- ② Use getByRole/getByTestId, not CSS/XPath
- ③ Use await expect(loc) — not expect(await)
- ④ Mock external APIs with page.route()
- ⑤ Each test gets own { page } fixture

02. Wait Functions

- waitForNetworkIdle(page, { idleTime })
- waitForSelectorStable(locator, { stableFrames })
- waitForApiResponse(page, urlPattern)
- All replace page.waitForTimeout()
- All use verified Playwright v1.x APIs

03. Locator Hierarchy

- HIGH getByRole / getByTestId / getByLabel
- MED getByText / getByPlaceholder / getByAltText
- Use filter() to narrow dynamic lists
- Use or() for unpredictable elements
- Never: CSS classes, XPath chains, nth() alone

04. Mock Patterns

- route.fulfill({ status, json }) — control response
- setTimeout delay — test loading states
- route.continue() — pass-through for stable APIs
- route.fetch() — modify real response
- Use fixtures for repeated mock setups

[DOCS] **Official Docs:** playwright.dev/docs/actionability • playwright.dev/docs/locators • playwright.dev/docs/mock • playwright.dev/docs/best-practices • playwright.dev/docs/network

Instagram: @shamshutalks

YouTube: ShamshuTalks

LinkedIn: ShamshuTalks